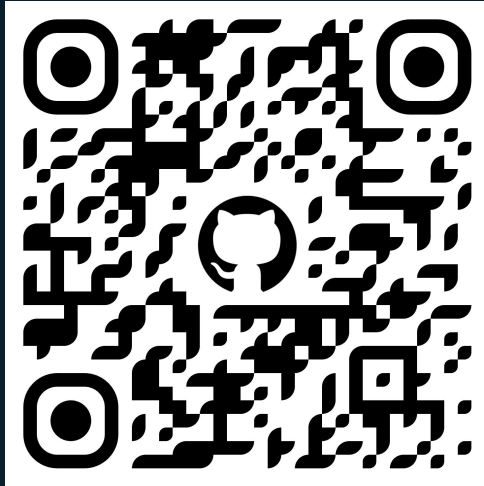


Resource



github.com/ro-witthawin/TBS-BI303-2026

TBS-BI303-2026



Class schedule

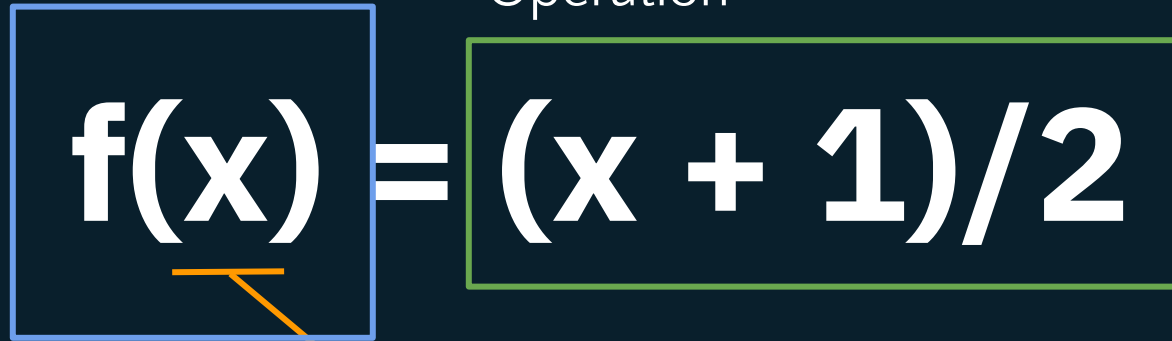
Date	Topic	Material
2026-08-03	Intro to Programming Language	Day01
2026-08-10	Basic String & List	Day02
2026-08-17	Function	Day03
2026-08-24	Loop	Day04
2026-08-31	Dictionary	Day05
2026-09-07	Class Object	Day06
2026-09-14	Freq Library	Day07
2026-09-21	Final Exam	

“Function”

$f(x)$

Function

Operation



The diagram shows the function definition $f(x) = (x + 1)/2$. The expression $f(x)$ is enclosed in a blue rectangular box, and the expression $(x + 1)/2$ is enclosed in a green rectangular box. An equals sign is placed between the two boxes. An orange line points from the word "Input" to the variable x in the function notation. A short orange horizontal line is positioned directly below the x .

$$f(x) = (x + 1)/2$$

Input

Function is more than just Math

```
def imageGenerate(prompt):
```

```
    new_prompt = explainPrompt(prompts)
```

```
    init_image = text2image(new_prompt)
```

```
    reconstruct_image = diffusion(init_image)
```

```
    final_image = upscale(reconstruct_image)
```

```
    return final_image
```

Sometime we don't need output

```
def imageGenerate(prompt):
```

```
    new_prompt = explainPrompt(prompts)
```

```
    init_image = text2image(new_prompt)
```

```
    reconstruct_image = diffusion(init_image)
```

```
    final_image = upscale(reconstruct_image)
```

```
    return final_image saveImage(final_image)
```

Function without return

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

```
def eq1(x):  
    if x > 0:  
        return x + 1  
    elif x < 0:  
        return x - 1
```

```
y = eq1(0) # y= None
```

```
def eq1(x):  
    if x > 0:  
        output_value = x + 1  
    elif x < 0:  
        output_value = x - 1  
    return output_value
```

```
y = eq1(0) # Error
```

Function without return

```
def imageGenerate(prompt: str):
```

```
    new_prompt = explainPrompt(prompts)
    init_image = text2image(new_prompt)
    reconstruct_image = diffusion(init_image)
    final_image = upscale(reconstruct_image)
    saveImage(final_image)
```

```
imageGenerate("Bird with Pen") # Return None
```

None Type

None

Any Function without return any value, it will return None

Example return None

```
score = 10
```

```
print(f"Score : {score}") # Display "Score : 10"
```

```
x = print(f"Score : {score}") # Display "Score : 10"  
print(x) # Display "None"
```

Empty List

[]



Empty List

`[]` → **None**



How to check None

x is None

x is not None

Basic List

[0, 1, 2, 3, 4]

Index

[0]

[1]

[2]

[3]

[4]

Negative
Index

[-5]

[-4]

[-3]

[-2]

[-1]

Range Function

range(start, stop, step)

`range(5)` # [0, 1, 2, 3, 4]

`range(1, 5)` # [1, 2, 3, 4]

`range(0, 50, 10)` # [0, 10, 20, 30, 40]

How to check None: Example



```
def purchasing Iphone(income=None):  
    if income is None:  
        return False  
  
    if income > 30000:  
        return True  
    else:  
        return False
```

Hand-on 20 min without AI

```
def iphone_plan(  
    iphone_price: float,  
    monthly_income: float,  
    Installment_months: int | None = None):
```

```
    pass # do_something
```

```
    return n_month, monthly_payment, status
```

```
# status = "Affordable" or "Insufficient"
```

```
# months_needed : int, monthly_payment : float
```



Level : ★★☆☆☆

iphone = 47,000 / monthly-income / installment

installment is None?

True

False

size n-month = 1

if monthly-income $\geq$ iphone-price
→ return, "Affordable", iphone-price

else
→ return, "Insufficient", iphone-price

size n-month = 3/6/12/24

monthly-price = $\frac{\text{iphone-price}}{\text{n-month}}$

if monthly-income $>$ monthly-price
→ return, monthly-price

else
→ return, monthly-price

def iphone-plan (iphone-price, monthly-income, installment-month):

if installment-month is None: # trick

if monthly-income $>$ iphone-price:
return "Affordable", iphone-price
else:

return "Insufficient", iphone-price

else:

def iphone-plan (iphone-price, monthly-income, installment-month):

if installment-month is None: # trick

if monthly-income $>$ iphone-price:
return 1, "Affordable", iphone-price

else:
return 1, "Insufficient", iphone-price

else:

price-per-month = iphone-price / installment-month
n = installment-month

if monthly-income $>$ price-per-month:
return n, "Affordable", price-per-month

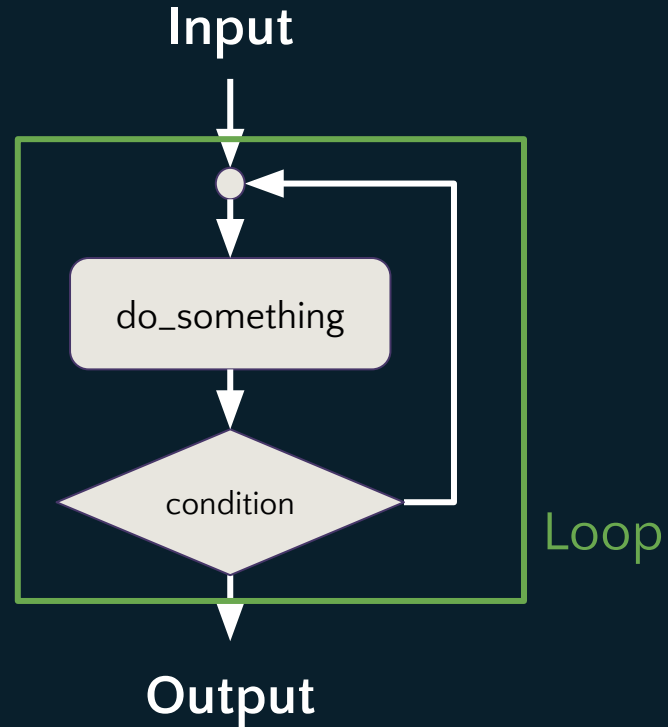
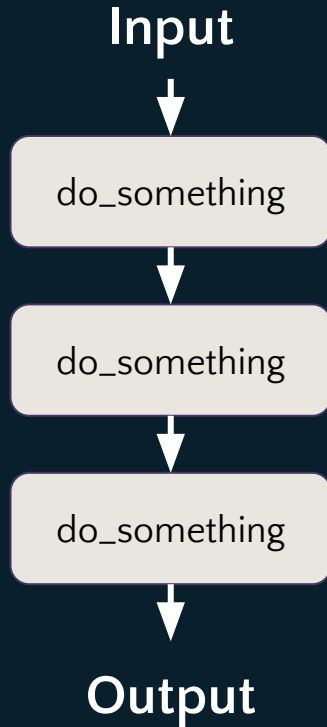
else:
return n, "Insufficient", price-per-month

function $\neq$ Condition

String/List/Loop

“Loop”

Loop Meaning



Loop Example : Sum

Ex: Sum of salary in this month when we have $X = [x_1, x_2, x_3, x_4, \dots, x_n]$

```
def sum_of_salary(X: list)
```

```
    n = len(X)
```

```
    salary = 0
```

```
    salary = salary + X[0]
```

```
    salary = salary + X[1]
```

```
    ...
```

```
    salary = salary + X[n-1]
```

```
    return salary
```



We can solve with
Loop n time!

For-Loop

```
def sum_of_salary(X: list)
    n = len(X)
    salary = 0
    for i in range(n):
        salary = salary + X[i]
    return salary
```

While-Loop

```
def sum_of_salary(X: list)
    n = len(X)
    salary = 0
    i = 0
    while (i < n):
        salary = salary + X[i]
        i += 1 # i = i + 1
    return salary
```

For-loop-in-List

```
for x_i in X:  
    print(x_i)
```

["a", 1, "b", 3, 4]



x_0



x_1



x_2



x_3



x_4

For-loop-in-String

```
for x_i in "Hello World":  
    print(x_i)
```

H -> x_0

e -> x_1

l -> x_2

l -> x_3

o -> x_4

-> x_5

W -> x_6

o -> x_7

r -> x_8

l -> x_9

d -> x_10

For-loop with index

```
i = 0
for x_i in "Hello World":
    print(f"{i} : {x_i}")
    i += 1
```

H -> x_0

e -> x_1

l -> x_2

l -> x_3

o -> x_4

-> x_5

W -> x_6

o -> x_7

r -> x_8

l -> x_9

d -> x_10

For-loop with enumerate

```
for i, x_i in enumerate("Hello World"):
    print(f"index_{i} : {x_i}")
```

For-loop with condition

Ex: Find finally cost when item costs less than 100 baht, add 10% to the transfer.

```
X = [10, 20, 100, 250, 400] # price
cost = 0
for price_i in X:
    if price_i < 100:
        cost += price_i + (price_i * 10 / 100)
    else:
        cost += price_i
print(cost)
```

Loop in Loop

The employee salaries in a company are stored as `salary_i_j`, where `i` represents the `i`th employee in the company and `j` represents the month. The data is stored as follows:

```
[  
  [salary_0_0, salary_0_1, ..., salary_0_5],  
  [salary_1_0, salary_1_1, ..., salary_1_4]  
]
```

Each employee's monthly salary varies due to their commission, making the amounts different every month. At the end of the year, we need to prepare the withholding tax certificates (50 Tawi). A 3% tax must be withheld for anyone whose salary exceeds 15,000.

How much tax does each person pay in total?

Level : ★★★★★

[[15000, 25000, 30000, 0],
 [20000, 50000, 200000],
 [15000, 12000, 16000, 14000]
]] $\Rightarrow$ input

Loop - for
 salary-0 > 15000
 salary-1
 salary-2
 employee-0
 employee-1
 employee-2

if salary > 15000
 duty = salary * $\frac{3}{100}$

def duty-by-employee (employee-j):

employee-duty-j = 0

for salary-i in employee-j:

if salary-i > 15000:

duty-month = salary * 3 / 100

employee-duty-j += duty-month

$\Rightarrow$ duty
 employee-0

[15000, 25000, 30000, 0]
 employee-0 $\Rightarrow$ [duty-by-employee] $\rightarrow$ duty

employee-0 $\Rightarrow$ duty-by-employee - duty-0

for employee-j in employee-all:
 duty-j = duty-by-employee (employee-j)